Guia de implantação — Brasil Fiscal (GitHub + Cloudflare + Supabase)

Este guia leva o projeto do repositório até um site funcionando na internet, usando só três serviços e **nenhum servidor para administrar**:

Peça	Papel
GitHub	Guarda o código. Cada mudança na branch `main` publica o site automaticamente.
Cloudflare (Workers)	Roda o site e a verificação diária das fontes.
Supabase	Banco de dados e login de usuários (já criado e com as tabelas prontas).

Siga os passos **na ordem, do 1 ao 17**: cada passo depende dos anteriores. Cada passo diz onde clicar ou o que digitar e como saber que deu certo. Nada precisa ser instalado no seu computador.

> **Sobre nomes de botões:** Cloudflare e Supabase mudam a aparência dos sites de vez em quando. Se um botão tiver nome um pouco diferente, procure o equivalente — o caminho do menu costuma ser o mesmo.

Regra de ouro: senhas, chaves e tokens **nunca** vão para o GitHub, prints ou chats. Eles ficam só na sua ferramenta/gerenciador de senhas e no painel da Cloudflare.

---

1. Conferir o que você precisa ter

- ☐ Conta no **GitHub** com acesso ao repositório `diellyr/Projeto_brasil_fiscal`.
- ☐ Conta na **Cloudflare** (https://dash.cloudflare.com/sign-up). Veja a observação do plano abaixo.
- ☐ Acesso ao **Supabase** (https://supabase.com/dashboard) com o projeto `Projeto_brasil_fiscal_db`.
- ☐ Um **aplicativo autenticador** no celular (Google Authenticator, Microsoft Authenticator, Authy...).

- [ ] Um **gerenciador de senhas** ou a sua ferramenta própria para guardar as variáveis.
- [ ] (Opcional) Um **domínio** próprio. Sem domínio, o site usa um endereço gratuito `*.workers.dev`.

⚠️ **Plano da Cloudflare.** O plano gratuito limita cada requisição a **10 ms de CPU**, o que costuma ser pouco para páginas do Next.js e pode causar o erro "Worker exceeded CPU time limit" (código 1102). O plano **Workers Paid** (cerca de US$ 5/mês) permite até 30 s de CPU e é o recomendado para uso real. Dá para começar no gratuito para testar e trocar depois em *Workers & Pages → Plans*.

✅ *Deu certo quando:* você consegue entrar nos três sites (GitHub, Cloudflare e Supabase).

---

2. Entender o que já está pronto (não refazer)

- O código está na branch `main` do GitHub, já preparado para a Cloudflare (`wrangler.jsonc`, `open-next.config.ts`, `cloudflare/worker.ts`).
- O banco do Supabase **já tem todas as tabelas criadas**. Ele está vazio de dados — normal: os dados entram conforme as pessoas consultam o site.
- A verificação diária das fontes (06:00 UTC = 03:00 em Brasília) já está configurada no código; ela começa a funcionar sozinha depois do deploy.

Nada a fazer neste passo; só leia.

---

3. Obter os valores das variáveis

A aplicação precisa de 6 variáveis. Duas são públicas e já vêm prontas; as outras quatro você obtém agora. Anote cada valor na sua ferramenta/gerenciador de senhas — você vai colar na Cloudflare nos passos 5 e 6.

Já prontas (públicas):

Variável	Valor
`NEXT_PUBLIC_SUPABASE_URL`	`https://tyhhowdudxwzowqhheng.supabase.co`
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	`sb_publishable_YlrayFVxs1tZGyAojLUfaw_FpxZehHe` (chave **pública**, pode aparecer no navegador)

Você precisa obter:

Variável	Como obter
`SUPABASE_SERVICE_ROLE_KEY`	Supabase → Project Settings → API Keys → `service_role` → Reveal
`CREDENTIALS_ENCRYPTION_KEY`	texto aleatório gerado (receita no passo 3.2)
`CRON_SECRET`	texto aleatório gerado (receita no passo 3.3)
`NEXT_PUBLIC_SITE_URL`	`https://` + o endereço final do site, sem barra no final (só será conhecido no passo 5)

3.1 `SUPABASE_SERVICE_ROLE_KEY` ⚠️ secreta

1. Supabase → abra o projeto → **Project Settings** (engrenagem, canto inferior esquerdo) → **API Keys**. Atalho: https://supabase.com/dashboard/project/tyhhowdudxwzowqhheng/settings/api-keys
2. Procure a chave `service_role` (em telas novas: **Secret keys** → `default`). Clique em **Reveal** e copie.

Esta chave ignora todas as proteções do banco. Nunca no GitHub, em prints ou chats.

3.2 `CREDENTIALS_ENCRYPTION_KEY` ⚠️ secreta e insubstituível — cifra as credenciais das fontes guardadas no banco.

> ⚠️ **O valor é o texto GERADO, não o comando.** A tabela do passo 3 mostra o comando `openssl rand -base64 32` só como receita; se você colar esse comando como valor, o site recusa (o `/api/health` mostra `invalid`).

Escolha **uma** forma de gerar:

- **No celular ou sem terminal (mais simples):** abra o gerador de senhas do seu gerenciador (Bitwarden, 1Password, o do Google/Apple...), gere uma senha **aleatória de 40 caracteres ou mais**, **sem espaços**, e use-a como valor.
- **No computador, no navegador:** F12 → aba **Console** → cole e dê Enter:

  ```
  btoa(String.fromCharCode(...crypto.getRandomValues(new Uint8Array(32))))
  ```

  (44 caracteres terminando em `=`.)
- **Com terminal:** `openssl rand -base64 32`

✅ O valor correto parece algo como `aB3kP9zQ7mW2xR5tY8uI1oL4...` — um texto sem sentido, **nunca** uma frase ou comando. **Faça backup em dois lugares**: se for perdido, as credenciais salvas ficam ilegíveis e precisarão ser cadastradas de novo.

3.3 `CRON_SECRET` — protege a rota da tarefa diária. É qualquer texto aleatório de **32 caracteres ou mais, sem espaços** (o mesmo gerador de senhas serve). **O valor é o texto gerado, não o comando** `openssl rand -hex 32`. No navegador (computador): `[...crypto.getRandomValues(new Uint8Array(32))].map(b=>b.toString(16).padStart(2,'0')).join('')`

3.4 `NEXT_PUBLIC_SITE_URL` — será definido no passo 5.4, quando você souber o endereço do site.

O mesmo conjunto está em `docs/VARIAVEIS.env` para conferência. Formato: `NOME=valor`, **sem aspas e sem espaços** ao redor do `=`. Não crie a variável `BRASIL_FISCAL_DEV_FIXTURES` (liga dados falsos de teste; o app recusa em produção).

---

4. Conferir o código no GitHub

1. Abra https://github.com/diellyr/Projeto_brasil_fiscal
2. Confirme que a branch selecionada é `main` e que existem os arquivos `wrangler.jsonc`, `open-next.config.ts` e a pasta `cloudflare`.

✅ *Deu certo quando:* você vê esses arquivos na raiz do repositório.

---

5. Criar o site na Cloudflare, ligado ao GitHub

5.1 Conectar o repositório

1. Cloudflare Dashboard (https://dash.cloudflare.com) → **Workers & Pages** → **Create** (ou **Create application**).
2. Escolha **Import a repository** (Workers com Git) e, se pedir, **Connect GitHub** → autorize o app *Cloudflare Workers and Pages* para o repositório `diellyr/Projeto_brasil_fiscal`.
3. Selecione o repositório `Projeto_brasil_fiscal` e clique em **Next**.

5.2 Configurar o build

Preencha **exatamente**:

Campo	Valor
Project name / Worker name	`brasil-fiscal` (precisa ser igual ao `name` do `wrangler.jsonc`)
Production branch	`main`
Root directory	deixe vazio (raiz)
Build command (*Comando de compilação*)	`npx opennextjs-cloudflare build`
Deploy command (*Comando de implantação*)	`npm run cf:release`

> ⚠️ **Não troque os dois campos.** `npx opennextjs-cloudflare build` vai em *Comando de compilação* (ele compila o site) e `npm run cf:release` vai em *Comando de implantação* (ele publica e envia os segredos). Se `npm run cf:release` for colocado no campo de compilação, o build falha logo depois de `> wrangler deploy && node scripts/sync-secrets.mjs`, reclamando de pasta/arquivo do site não encontrado.

5.3 Variáveis (um único lugar: as variáveis de build)

Todas as variáveis são cadastradas **num só lugar**: as **variáveis de compilação**. O *Comando de implantação* do passo 5.2 envia automaticamente os segredos ao site em execução, então você **não** precisa cadastrar nada na outra tela.

Onde fica (painel em português): Cloudflare → **Workers & Pages** → **brasil-fiscal** → aba **Configurações** → role a página para baixo, passando por *Variáveis e segredos em tempo de execução*, *Associações* e *Observabilidade*, até o cartão **Compilação** (em inglês, **Build**). Dentro dele, abra **Variáveis e segredos de compilação** (*Build variables and secrets*) → **Adicionar variável**.

> ⚠️ **Não é** a seção *Variáveis e segredos em tempo de execução* (a primeira da página). Nela aparecem apenas duas variáveis `NEXT_PUBLIC_SUPABASE_*` que vêm do repositório: deixe-as como estão e não adicione nada ali — cada implantação substitui o conteúdo dessa seção pelo que está no `wrangler.jsonc`.

Adicione estas 6 (Name = nome, Value = valor). Use o tipo **Secret** para as secretas e **Text** para as públicas:

Name	Tipo	Value
`NEXT_PUBLIC_SUPABASE_URL`	Text	`https://tyhhowdudxwzowqhheng.supabase.co`
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Text	`sb_publishable_YlrayFVxs1tZGyAojLUfaw_FpxZehHe`
`SUPABASE_SERVICE_ROLE_KEY`	**Secret**	do passo 3.1
`CREDENTIALS_ENCRYPTION_KEY`	**Secret**	do passo 3.2
`CRON_SECRET`	**Secret**	do passo 3.3
`NEXT_PUBLIC_SITE_URL`	Text	valor provisório: `https://brasil-fiscal.workers.dev` (corrigido no passo 5.5)

As duas `NEXT_PUBLIC_SUPABASE_*` precisam estar aqui porque o build as grava no site (o navegador as recebe compiladas).

> ⚠️ Cole **só o valor gerado** (nunca o comando `openssl …`) e sem espaço ou quebra de linha no fim.

5.4 Primeiro deploy

Clique em **Save and Deploy** e aguarde 3–6 minutos. Acompanhe em **Deployments / Builds**.

✅ *Deu certo quando:* o build termina com "Success" e aparece o endereço do Worker, no formato `https://brasil-fiscal.SEU-SUBDOMINIO.workers.dev`. **Anote esse endereço.**

❌ *Se o build falhar:* abra o build → role o log até o primeiro texto em vermelho (veja o passo 16).

5.5 Corrigir `NEXT_PUBLIC_SITE_URL`

1. Edite a variável de build `NEXT_PUBLIC_SITE_URL` (passo 5.3) com o endereço real do passo 5.4, **sem barra no final**.
2. Refaça o build: **Deployments → Retry build** (ou envie qualquer commit para a `main`).

---

6. Conferir se os segredos chegaram ao site

Na primeira implantação, o `Deploy command` (passo 5.2) enviou os segredos do passo 5.3 ao Worker. Confirme:

1. Em **Deployments / Builds**, abra o build mais recente e procure no log a linha `[sync-secrets] segredos enviados ao Worker.` (e **não** a linha `FALTAM nas variáveis de build`).
2. Cloudflare → **Workers & Pages** → **brasil-fiscal** → **Configurações** → **Variáveis e segredos em tempo de execução** (a seção de cima): devem aparecer `SUPABASE_SERVICE_ROLE_KEY`, `CREDENTIALS_ENCRYPTION_KEY`, `CRON_SECRET` e `NEXT_PUBLIC_SITE_URL` como **Secret** ("Value encrypted").
3. ⚠️ Se, nessa mesma seção, existirem variáveis **do tipo "Variável/Text" com esses mesmos nomes** (criadas à mão antes), **apague-as** (ícone da lixeira) e faça **Retry build** — nomes repetidos impedem o envio.

✅ *Deu certo quando:* o log do build mostra `segredos enviados ao Worker` e o `/api/health` do passo 7 mostra `"ok": true`.

---

7. Abrir o site e conferir

Abra `https://brasil-fiscal.SEU-SUBDOMINIO.workers.dev`.

✅ *Deu certo quando:* a home do Brasil Fiscal carrega **sem** a faixa amarela "MODO DE DESENVOLVIMENTO".

Diagnóstico rápido: abra `https://SEU-SITE/api/health`. Ele mostra, sem revelar valores, quais variáveis o site enxerga e se o banco responde:

- `"ok": true` → tudo certo.
- `"missing": [...]` → essas variáveis não chegaram ao site em execução. Confirme que as variáveis estão em *Build* → *Variables and secrets* (passo 5.3), faça **Retry build** e procure no log `[sync-secrets] segredos enviados ao Worker` (passo 6).
- `"invalid": [...]` → a variável existe, mas o valor está no formato errado (por exemplo, o comando `openssl rand …` colado no lugar do valor gerado). Gere o valor de verdade (passos 3.2 e 3.3) e substitua.
- Variável aparece cadastrada no painel mas `false` aqui → faça **Retry build** e confira o log (passo 6).
- Todas `false` → nenhuma variável de execução foi lida: o envio de segredos do build não rodou: confirme o *Deploy command* `npm run cf:release` (passo 5.2) e o nome do Worker `brasil-fiscal`.
- `"database": {"ok": false, "error": "Invalid API key"}` → a chave está errada (copiou a chave `anon` no lugar da `service_role`, ou veio com

espaço/quebra de linha).

❌ *Se aparecer erro 1102 / "exceeded CPU time limit":* é o limite do plano gratuito (passo 1). Assine o **Workers Paid**.

---

8. Autorizar o endereço do site no Supabase

1. Supabase → **Authentication** → **URL Configuration**.
2. **Site URL:** o endereço do passo 5.4.
3. **Redirect URLs** → *Add URL* → `https://brasil-fiscal.SEU-SUBDOMINIO.workers.dev/**`
4. **Save**.

✅ *Deu certo quando:* os campos continuam preenchidos ao recarregar a página.

---

9. Criar o primeiro administrador

9.1 Criar o usuário

1. Supabase → **Authentication** → **Users** → **Add user** → **Create new user**.
2. Informe e-mail e senha forte (12+ caracteres) e marque **Auto Confirm User**. Clique em **Create user**.

9.2 Torná-lo administrador

1. Supabase → **SQL Editor** → **New query**.
2. Cole (trocando pelo e-mail do passo anterior) e clique em **Run**:

```
update public.profiles set role = 'admin' where email = 'seu@email.com';
```

✅ *Deu certo quando:* aparece "1 row affected" (ou similar). Se aparecer 0 linhas, o e-mail está diferente do usuário criado — confira em *Authentication → Users*.

9.3 Conta provisória de administrador e acesso sem MFA

O banco foi preparado com uma conta de administrador provisória: **e-mail** `ti@local`. A senha inicial foi combinada por conversa e **não é registrada neste guia nem no repositório** (nunca guarde senhas no GitHub). O menu **Admin** no cabeçalho do site leva a `/admin` (que pede login). Para entrar **sem MFA** por enquanto:

1. Na Cloudflare, em **Configurações → Compilação → Variáveis e segredos de compilação**, adicione `ADMIN_REQUIRE_MFA` com o valor `0` e faça **Tentar novamente** no build. (Para exigir MFA de novo, apague a variável e refaça o build.)
2. Entre em `https://SEU-SITE/login` com `ti@local` e a senha inicial, e clique em **Admin**.

> 🔴 **Troque a senha imediatamente e ligue o MFA assim que possível.** Uma conta administradora com senha simples e sem MFA, num site público, dá controle total do painel (fontes, credenciais, logs) a quem adivinhar a senha. Para trocar a senha, no **SQL Editor** do Supabase (use uma senha longa e única no lugar de `NOVA_SENHA_FORTE`): `update auth.users set encrypted_password = extensions.crypt('NOVA_SENHA_FORTE', extensions.gen_salt('bf')) where email = 'ti@local';`

10. Ligar a verificação em duas etapas (MFA)

Em produção, o painel `/admin` **exige** MFA.

1. Supabase → **Authentication** → **Sign In / Providers** (ou **Multi-Factor**) → ative **TOTP** → salve.
2. Abra `https://SEU-SITE/login` e entre com o e-mail e a senha do passo 9.
3. Abra `https://SEU-SITE/login/mfa`. Aparecerá um **QR code**.
4. No celular: autenticador → **+** → **Escanear QR code**.
5. Digite o código de 6 dígitos e confirme.
6. **Guarde o "código secreto" em texto** que aparece junto do QR no gerenciador de senhas (serve se você perder o celular).

✅ *Deu certo quando:* `https://SEU-SITE/admin` abre o painel (menus Fontes, Saúde, Logs, Cache, Dados, Auditoria...).

---

11. Cadastrar as chaves das fontes de dados

Tudo dentro do app, em `https://SEU-SITE/admin/fontes`.

11.1 Portal da Transparência (necessária para despesas, benefícios e contratos)

1. Gere a chave grátis em https://portaldatransparencia.gov.br/api-de-dados/cadastrar-email (ela chega por e-mail).
2. No app: `/admin/fontes` → abra **Portal da Transparência** → seção **Credencial** → cole a chave → **Salvar**.
3. Clique em **Testar conexão**.

✅ *Deu certo quando:* o teste mostra "API disponível" e o status vira **ONLINE**.

11.2 SICONFI/Tesouro e IBGE — não precisam de chave. Só clique em **Testar conexão** em cada uma.

11.3 dados.gov.br (opcional) — token em https://dados.gov.br (perfil → *Token de acesso*); cadastre como no 11.1.

11.4 Receita Federal e outros CSVs (opcional) — `/admin/fontes` → **Adicionar fonte** → nome, endereço **https** do CSV e mapeamento das colunas. Endereços internos/privados são recusados por segurança.

---

12. Conferir a verificação diária (cron)

1. Cloudflare → **Workers & Pages** → **brasil-fiscal** → **Settings** → **Trigger Events** (ou **Triggers**) → **Cron Triggers**: deve listar `0 6 * * *`.
2. No dia seguinte, veja o resultado em **Observability / Logs**: procure por `[cron] health 200`.
3. Para testar na hora (opcional), troque `SEU-SITE` e `SEU_CRON_SECRET`:
   - **Windows (PowerShell):** `Invoke-RestMethod -Uri "https://SEU-SITE/api/cron/health" -Headers @{ Authorization = "Bearer SEU_CRON_SECRET" }`
   - **Mac/Linux:** `curl -H "Authorization: Bearer SEU_CRON_SECRET" https://SEU-SITE/api/cron/health`

✅ *Deu certo quando:* a resposta traz algo como `{"tested":{...},"purged":...}`. Resposta `401` = `CRON_SECRET` diferente do cadastrado no passo 6.

---

13. Conferir tudo (checklist final)

- ☐ A home abre e **não** mostra "MODO DE DESENVOLVIMENTO".
- ☐ Pesquisar "Praia Grande" abre a página do município com indicadores.
- ☐ Cada número mostra **fonte** e **data** ("Fonte e período").
- ☐ `/admin/saude` lista as fontes com status (ONLINE/OFFLINE/NÃO CONFIGURADA).
- ☐ `/admin/dados` mostra entidades e fatos **aumentando** após as consultas.
- ☐ `/admin/logs` mostra as chamadas às APIs (se alguma falhar, o erro aparece aqui).
- ☐ O passo 12 retornou `200` e o Cron Trigger está listado.

> O código foi escrito pela documentação das APIs do governo e testado com dados de exemplo. Na primeira consulta real, alguma fonte pode responder num formato levemente diferente. Isso **não** gera dado inventado: a consulta

> falha com mensagem clara em `/admin/logs`. Se acontecer, guarde a mensagem (sem chaves!), a data/hora e a fonte, e abra uma *Issue* no repositório.

---

14. Importar dados por CSV (opcional)

Quando uma API do governo estiver instável, ou você já tiver os dados em planilha, importe um CSV por fonte. **O sistema usa o CSV antes de consultar a API**: para os entes, anos e períodos que o arquivo cobre, a API nem é chamada; para o resto, continua buscando nela.

1. Entre no painel (`/admin`) → **Fontes de Dados** → na fonte desejada (ex.: SICONFI), clique em **Importar CSV**.
2. Escolha **o que o arquivo contém** (ex.: "Despesas por função e subfunção") e clique em **Baixar modelo de CSV** para ver as colunas aceitas. Dica: abra o modelo no Excel/Planilhas, cole seus dados e salve como **CSV**.
3. Selecione o arquivo. Ajuste, se precisar, a **codificação** (Excel brasileiro costuma gerar ISO-8859-1), o **separador** e o **decimal** (`1.234,56` ou `1,234.56`).
4. Confira o **mapeamento das colunas**: a tela sugere pelo nome do cabeçalho; corrija o que for preciso. Se o arquivo não tem um campo (ex.: todas as linhas são "Município"), use o **valor fixo**.
5. Veja a **Conferência** (linhas válidas, problemas por linha) e clique em **Importar**. Acompanhe a barra de progresso.
6. O **Histórico de importações** mostra o resultado; o botão **Desfazer** apaga os dados daquele CSV e devolve esses períodos para a API.

Regras importantes:

- Código do ente: município = **7 dígitos** IBGE; estado = 2 dígitos ou sigla (SP); país = 1.
- A importação **substitui** os dados já salvos daquelas unidades (mesma fonte e mesmo conjunto), para não somar duas vezes.
- Cada número importado aparece no site com a origem "(CSV)" e a data da importação.
- Linhas com problema são recusadas **com o motivo**; as demais são gravadas.

---

15. Instalar como aplicativo (PWA)

O site pode ser instalado como aplicativo, sem loja:

- **Android (Chrome):** abra o site → menu ⋮ → **Instalar app** (ou use o botão **Instalar aplicativo** no rodapé do site).
- **iPhone/iPad (Safari):** botão **Compartilhar** → **Adicionar à Tela de Início**.
- **Computador (Chrome/Edge):** ícone de instalação na barra de endereço, ou o botão do rodapé.

O aplicativo usa sempre dados ao vivo: sem internet aparece uma página "Você está sem conexão" (os números nunca são guardados no aparelho, para não mostrar informação velha sem aviso). Atualizações do site chegam automaticamente.

---

16. Problemas comuns

16.1 O build na Cloudflare falhou — abra o build e leia o primeiro erro em vermelho:

- O log para logo depois de `> wrangler deploy && node scripts/sync-secrets.mjs`, ou reclama de `.open-next` / `assets` não encontrado: `npm run cf:release` foi colocado no campo **errado** (compilação). Ajuste o passo 5.2.
- "Worker name mismatch / name ... does not match": o nome do projeto precisa ser `brasil-fiscal` (passo 5.2).
- "NEXT_PUBLIC_..." ou erro de variável: confira as variáveis de **build** (passo 5.3).
- "Script too large": o plano gratuito limita o tamanho; use Workers Paid.

16.2 O site abre, mas mostra "Configuração necessária" ou dá erro ao pesquisar — abra `https://SEU-SITE/api/health` (passo 7) e siga o que ele indicar. As causas mais comuns: a variável foi cadastrada só em *Build* e não em *Variables and Secrets* do Worker (passo 6); o nome tem erro de digitação; ou o valor tem espaço/quebra de linha no fim. Corrija e faça **Deploy**. Os detalhes técnicos ficam em **Observability / Logs** do Worker.

16.3 Erro 1102 / "exceeded CPU time limit" — limite do plano gratuito; assine o Workers Paid (passo 1).

16.4 Mudei uma variável e nada mudou — altere em *Settings* → *Build* → *Variables and secrets* e faça **Retry build**: o build grava as `NEXT_PUBLIC_*` no site e reenvia os segredos ao Worker.

16.5 "Não foi possível entrar" em /login — confirme e-mail/senha em *Authentication* → *Users* (usuário confirmado) e o **Site URL** do passo 8.

16.6 Uma fonte aparece OFFLINE ou "credencial ausente/inválida" — em `/admin/fontes` clique em **Testar conexão** e leia a mensagem. "Credencial inválida": gere uma chave nova (passo 11.1). "Falha de conexão": a API do governo pode

estar fora do ar; o site segue mostrando o dado salvo, com aviso.

16.7 Esqueci/perdi o `CREDENTIALS_ENCRYPTION_KEY` — não há como recuperar. Gere outra (passo 3.2), atualize no passo 6, faça Deploy e **recadastre** as credenciais das fontes (passo 11).

16.8 Perdi o autenticador (MFA) — outro admin (ou o dono do projeto) remove o fator em Supabase → *Authentication* → *Users* → usuário → *Delete/Remove MFA factors*; depois refaça o passo 10.

16.9 Muitos acessos bloqueados (erro 429) — o limite de requisições por IP usa a memória de cada instância. Para um limite compartilhado, crie um banco gratuito em https://upstash.com (Redis) e cadastre `UPSTASH_REDIS_REST_URL` e `UPSTASH_REDIS_REST_TOKEN` no passo 6. Você também pode criar uma regra em Cloudflare → *Security* → *WAF* → *Rate limiting rules*.

---

17. Rotina do dia a dia

- **Atualizar o site:** qualquer mudança que chegar na `main` do GitHub é publicada automaticamente pela Cloudflare. Acompanhe em *Deployments*.
- **Voltar a uma versão anterior:** *Deployments* → versão antiga → **Rollback**.
- **Trocar uma variável:** altere em *Settings* → *Build* → *Variables and secrets* (passo 5.3) e faça **Retry build**.
- **Domínio próprio:** Worker → *Settings* → *Domains & Routes* → *Add* → *Custom domain* (o domínio precisa estar na Cloudflare). Depois atualize `NEXT_PUBLIC_SITE_URL` (execução e build) e o **Site URL/Redirect URLs** do Supabase.
- **Desativar uma fonte:** `/admin/fontes` → chave **Ativar/Desativar** (fica registrado em `/admin/auditoria`).
- **Dar acesso de admin a outra pessoa:** repita os passos 9 e 10 para o e-mail dela.
- **Backups:** guarde uma cópia das variáveis no gerenciador de senhas (principalmente `CREDENTIALS_ENCRYPTION_KEY`) e ative os backups do banco no Supabase (*Database* → *Backups*).

Glossário rápido

- **Cloudflare Workers:** serviço que executa o site sem você administrar servidor.
- **Deploy:** publicar uma versão do site. **Rollback:** voltar a uma versão anterior.
- **Variável de build × de execução:** a de build é lida ao compilar o site; a de execução é lida a cada requisição.
- **Secret:** variável cujo valor fica escondido depois de salvo.

- **Cron Trigger:** agendamento da Cloudflare (aqui, a verificação diária das fontes).
- **Migration:** script que cria/altera tabelas do banco (já aplicadas).
- **RLS (Row Level Security):** regras do banco que decidem quem lê/escreve cada linha.
- **MFA/TOTP:** segundo fator de login com código de 6 dígitos gerado no celular.
- **Fonte (data source):** API ou arquivo de dados públicos cadastrado no painel.

Onde ler mais

README · ARCHITECTURE · DATABASE · DATA_SOURCES · API · SECURITY